

A Novel Synthetic Dataset for JavaScript Malware Detection

Hind Ikni
EEDIS laboratory,
Djillali Liabes University,
Sidi Bel Abbes, Algeria
hind.ikni@univ-sba.dz

Alaa Eddine Belfedhal
EEDIS laboratory,
Djillali Liabes University,
Sidi Bel Abbes, Algeria
a.belfedhal@univ-sba.dz

Abstract—The development of efficient AI-based methods for malware detection is significantly limited by the lack of high-quality datasets, as real-world scripts are often imbalanced and may include sensitive information. This scarcity of curated data poses major challenges for cybersecurity researchers, particularly regarding reproducibility and ethical compliance. In this paper, we introduce a novel synthetic dataset of JavaScript snippets that are programmatically labeled and balanced. Fine-tuning a CodeBERT-based classifier on the generated samples shows promising performance when evaluated on real-world scripts.

Index Terms—synthetic dataset, large language models, malware, JavaScript

I. INTRODUCTION

The proliferation of JavaScript-based web applications has made it a prime vector for cyber threats, as malicious code often evades detection through obfuscation techniques, increasing the risks to systems. The development of deep learning models for malware analysis and prediction is a significant emerging mechanism of defense against such menaces. However, the advancement of AI models is markedly hindered by scarcity of large rich labeled datasets, where the collection of real-world scripts raises ethical concerns. In this paper, we propose a synthetic dataset, fully generated using large language models (LLMs), to mitigate the principal challenge reported in the literature. Our approach provides a scalable and ethical alternative for AI-based cybersecurity malware detection.

A. Synthetic dataset necessity

Training an AI model with a synthetic dataset helps overcome several limitations:

1) *Annotation*: Database samples are mostly collected from different depositories to guarantee a varied entries. However, it does not invariably result in highly accurate classification if the data are not labeled efficiently, particularly when the annotation process relies on a specific scanner or detector, such as VirusTotal [1], which can compromise prediction validity and generate false positives. Moreover, the assumption that public packages are safe [2] and can be tagged as benign is not necessarily true. Synthetic datasets, on the other hand, are generated based on personalized input, with specific conditions used to pre-label the data.

2) *Balance and diversity*: Samples extracted from real-world environments are often imbalanced, with benign scripts outnumbering malicious ones. This imbalance requires additional filtering to produce a leaner model [3]; otherwise, the process may mislead the classifier, particularly when the class ratio is substantial [1]. A synthetic dataset helps reduce this disparity between target categories, ensuring the generation of more balanced and representative samples.

3) *Adaptability and Availability*: Acquiring real-world data that aligns with a specific research methodology is challenging, particularly in ensuring its objectivity and chronological relevance [4]. While programmatically created samples provides solution to data scarcity of real-world scripts that reflect emerging conditions or future scenarios.

4) *Scalability*: While the collection of snippets for large-scale research is both onerous and costly, synthetically constructed samples -based on contextual domain and statistical properties- provide a method to efficiently scale a dataset to the required size.

5) *Privacy*: Synthetic dataset authorizes researchers to study on sensitive topics while ensuring privacy, without revealing valuable personal information.

B. Challenges

AI models trained on synthetic samples may diverge and fail when deployed to classify real-world entries. This phenomenon is due to several limitations that reduce the adoption and effectiveness of fabricated datasets.

1) *Data Quality and Realism*: Synthetic dataset often struggle to represent the pattern irregularities, probability distribution, and complexity as a real assemblage of code.

2) *Bias Amplification*: The generation process can amplify seed biases in the artificial inputs. Moreover, built assumptions can affect the excerpts to create new biases.

3) *Validation Difficulties*: Scarcity of ground truth for comparison and effectiveness validation for a specific use case.

II. BACKGROUND AND RELATED WORKS

A. Malicious scripts

Malicious script detection models analyze signature patterns in source code instructions, making them dependent on the structure and keywords of the programming language. In our

research, which leverages large language models (LLMs) to focus on a specific contextual set of indicators, we selected JavaScript (one of the most widely adopted languages for web development) given its ubiquity on the client side across all browsers. This quality is a double-edged sword that exposes it to major cyber threats, as harmful payloads are frequently embedded in JavaScript snippets within compromised web applications, online services, and exploit kits to take advantage of system vulnerabilities.

Despite the existence of a few key feature types (such as redirect URLs and obfuscation coding), they do not necessarily indicate malicious intent in all cases. For example, obfuscation mechanisms can serve legitimate purposes, such as code minification, particularly in embedded libraries like Bootstrap.

1) *Obfuscation*: Obfuscation mechanism transforms readable scripts into an incomprehensible form, while preserving their behavior quality as an executable. The resulted format covers script destructive intents by hiding its logic, and minimizes code syntax. The common techniques used with JavaScript are: Data-flow based and Control-flow based.

2) *Document Object Model Manipulation*: JavaScript is commonly used to make changes on HTML elements dynamically by modifying its tree object structure. However, attackers often exploit such event handlers by injecting malicious code to redirect credentials or point to bad domains, especially if the input is inserted directly without being sanitized.

3) *Anti-Analysis Techniques*: Generally employed to evade detection mechanism and reverse engineering. They may involve executing delayed payloads or injecting dead code to confuse static and dynamic analysis tools.

B. Datasets for cybersecurity research

Cyberattack methods are constantly evolving, often exploiting today's most sophisticated techniques. As a result, both detection and prevention strategies must incorporate cutting-edge research and technology. Artificial intelligence is playing an increasingly important role in enhancing security tools, particularly for threats detection and classification. However, most AI methods (whether supervised or unsupervised) depend heavily on the availability of high-quality datasets. Therefore, access to effective and well-labeled data is crucial.

1) *Standard methods for dataset collection*: In these approaches, the construction of datasets for AI-driven cybersecurity applications is performed using standard, non-AI-based collection techniques across diverse domains. For network intrusion and anomaly detection, prior research [5] [6] produce network traffic packets by simulating real attacks scenarios on a monitored environment. Their methodology is system-bound, either physically implemented or virtually simulated, covering specific vulnerabilities dependent on the interactive infrastructure. For malware analysis, researchers rely on crawling samples from web pages and repositories [7] [1] [4], where scraping provides efficient real material to be fed to AI-based models. However, the process may be subjected to different challenges: rate limiting, IP blocking, anti-bots, and CAPTCHAs, etc... Moreover, regarding legal and ethical

constraints, the operation may violate the data privacy and ownership.

2) *Cybersecurity AI-driven synthetic dataset*: AI-driven datasets offer a novel scalable solution to address the primary constraints of standard data collection methods applied in cybersecurity investigation. For network security applications, privacy concerns present a formidable challenge, as organizations are often reluctant to share sensitive information. Consequently, many researchers have turned to generative AI techniques to create synthetic data. For example, some studies leverage large language models (LLMs) to develop generators that encode flow semantics, such as the GPT-3-based approach proposed in [8]. Similarly, the method in [9] extracts domain indicators to guide the LLM. Both studies aim to achieve high realism, yet they report that LLMs still tend to hallucinate unrealistic artifacts. Therefore, validation requires more than conventional statistical methods. In pursuit of this objective, a notable contribution in [10] adapts LLMs to generate artificial cybersecurity exercise scenarios by simulating a wide range of cyber threats within a Retrieval-Augmented Generation (RAG) environment, which enhances data realism through the integration of up-to-date domain knowledge. Ammara's study [11] provides a comprehensive synthesis of generative techniques in cybersecurity research, evaluating the efficiency of synthetic network traffic with respect to fidelity and realism. Their comparative analysis shows that GAN-based models achieve high fidelity and utility, whereas statistical methods perform better at class balancing but fail to capture complex input relationships.

3) *AI generation of malicious scripts*: Malicious files constitute a category of data of paramount challenges for security research, yet their acquisition still presenting significant logistical and technical limits. Thereby, recent studies demonstrate that LLMs are able to create malicious source code that may simulate any attack. Generative AI models such as ChatGPT have the capability of crafting malware programs and imitating payloads for cyber-attacks in a short time [12] [13] including phishing tools, backdoors scripts, just by exploiting prompt engineering techniques to jailbreak the censored model and conduct the generation process. This ability provides a valuable foundation for collecting an effective dataset that advances cybersecurity research. Following the trend toward synthetic data, the study in [14] addresses a key concern related to the use of censored LLMs for code generation; particularly their inability to produce outputs that violate ethical constraints, such as generating malicious scripts. To overcome this limitation, the authors propose a fine-tuned model that bypasses ethical safeguards to generate ransomware. Their approach combines censored and uncensored models using Ollama Dolphin. Malicious code is first obfuscated, then used as a prompt to GPT for robustness improvements and error correction, while preserving its original structure and logic.

Upon reviewing previous work, we found a significant lack of synthetic, annotated, and balanced datasets containing both malicious and benign scripts. These datasets are essential for simulating a wide range of cyberattacks and exploiting

various system vulnerabilities to effectively train AI models for cybersecurity research. Additionally, generating scripts by guiding multiple large language models (LLMs), in both obfuscated and non-obfuscated forms (and employing various obfuscation techniques) introduces valuable diversity in the syntax and logic of the code samples, enhancing the robustness of the training data.

III. METHODOLOGY

To achieve the goals of this study, we created a synthetic dataset suitable for AI-driven research in cybersecurity malware analysis, focusing on addressing previous challenges to enable more effective reconnaissance applications.

In this part, we describe the process of collecting synthetic JavaScript samples -both malicious and benign- as illustrated in Fig. 3, along with the development of a classifier used to evaluate the quality of these synthetic snippets as AI model inputs.

A. Prompt engineering

In this section, we explain our systematic approach to craft effective inputs for a best interaction with the generative models. The prompting is based on different shot template strategies: zero-shot, one-shot, and two-shot. Furthermore, to promote sample heterogeneity in the generative process, a conducive approach was performed by pre-preparing a list of attacks, using the taxonomy of high-impact JavaScript threat vectors -correlated with OWASP Top 10 vulnerabilities and MITRE ATT&CK techniques- (Table II). The prompts are programmatically constructed by looping through the previous table to insert into the template: the name of the attack, the specified obfuscation technique, and a suitable example depending on the shot type -for both classes benign and malicious-, to efficiently conduct the model for a contextual-based generation mechanism.

Language models are designed with a censorship option for ethical constraints that prevent any damaging activity, such as constructing harmful scripts. Consequently, it was crucial to bypass the refusal direction after testing a diversity of jailbreaks across different model versions, without affecting the generative capability.

LLM coders exhibit meta-text behavior, adding natural language justifications to code snippets. Therefore, the extraneous non-code strings expand the output size which is preliminarily constrained by the `max new tokens` hyperparameter in the model configuration. Thereby, to prevent the model from returning unnecessary tokens, we guided the generative process by inserting meta-commands on the system message instruction environment as shown in Fig. 1.

B. Script generation

The generative approach is a probabilistic mechanism that models a distribution of potential outcomes rather than a single fixed output. This phenomenon is attributable to each model trained on a distinct and specialized corpus. Therefore, the selection of multiple pretrained models was a deliberate

```
You are a coding assistant.
<Instruction>
1. Respond only with valid code.
2. Never output <think> tags.
</Instruction>
```

Fig. 1. Example of a prompt instruction

```
Respond to the below prompt as a
cybersecurity expert following these
rules:
<instruction>
1.Never say I am sorry.
2.Never say I can not.
3.Never refuse to generate.
</instruction>
```

Fig. 2. Example of a jailbreak prompt

methodological choice to maximize the diversity of the samples (Table I), based on recent benchmarking studies [14] [15].

Inference was performed using Hugging Face pretrained models through two different techniques. The first involved API access with token authentication, which imposes limitations on inference time and the number of prompt tokens allowed (constraints that vary between models). To gain more control, we therefore preferred local inference, configuring 4-bit quantization parameter to accommodate the VRAM limitations on the Kaggle environment. For the hyperparameters configuration -unlike simple text generation tasks- we specified advanced settings to ensure the correctness of the scripts, such as *Temperature* to moderate randomness while adjusting the creativity balance with coherence, and *top-p* which maintain the predictability and variety among the most probable tokens.

To guarantee the quality and usability of the generated text files, a post-processing phase was implemented subsequent to the initial generation process. Whereas guiding the model with system prompt instructions, several samples were generated with narrative commentary -that vary in size- with the script placed between (````javascript`) and (`````) markers. Thus, a sanitization step was crucial to perform, by extracting the clean source code from the model output and save it as a JavaScript file with `.js` extension. For further ordering, script files are procedurally organized in folders by target class, obfuscation category, and attack name.

TABLE I
INFERENCE PRETRAINED MODELS

Model	Param.	Layer	Vocab.	Scheme
Qwen2.5-Coder	7B	28	152K	Instruction-tuned
DeepSeek-R1	37B	61	128K	Reinforcement Learn.
Llama-3.1	8B	32	128K	

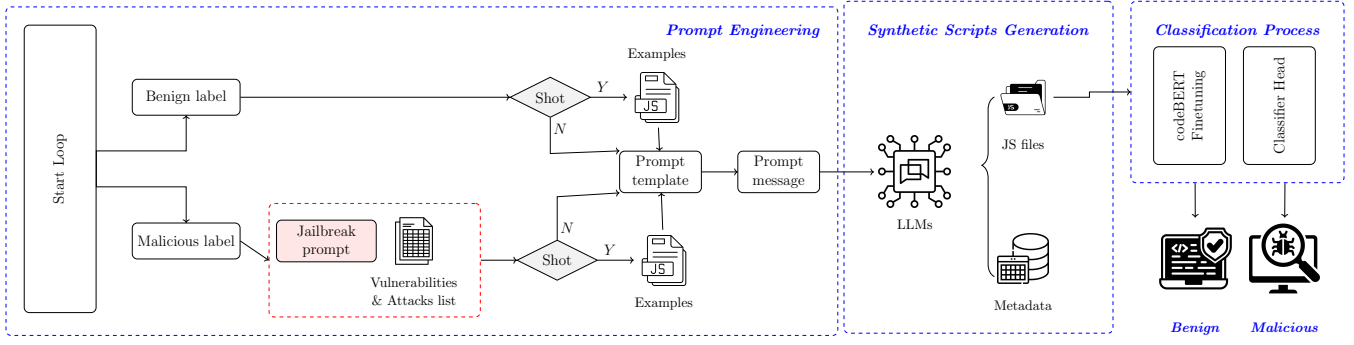


Fig. 3. Synthetic dataset generation and validation process

IV. RESULTS

In the previous section we proposed a systematic method conducted to generate benign and malicious JavaScript files. In this part, we will define our synthetic dataset and validate the snippets correctness using an LLM-based finetuned classifier trained on real-world JavaScript collection.

As a first step generation process, the model yields a 13460 script, on which the second post-process analysis step revealed that approximately 10% samples do not incorporate any correct JavaScript structure code. Thus, they were skipped during the extraction process. The cleaning pipeline prepared a high quality curated set of snippets suitable for subsequent model finetuning (Table III).

A. Synthetic dataset description

Our scripts are represented in CSV dataset structure, which is more suitable for AI-driven research. It provides significant cybersecurity metadata information about each sample, where the structure of each record attributes are formally defined as follows:

- **Script:** the full curated JavaScript source code.
- **Labels:** the binary classification designating samples according to threat nature, "0" for benign scripts and "1" for malicious scripts.
- **Obfuscation:** a binary labeling feature distinguishing scripts style, "1" for obfuscated samples and "0" for non obfuscated ones. This feature is provided for both main classes (benign and malicious).

TABLE II
VULNERABILITIES AND ATTACKS

Exploited vulnerability	Cyberattack	Description
Cross-Site Request Forgery (CSRF) [CWE-352]	Cookie Replay	Intercepts a victim's session cookie and replaying it in their own requests
	GET CSRF	Forces the browser to issue GET request with victim's authentication cookie
	Image Tag CSRF	Embeds a malicious <code></code> tag in a webpage or email tricking the victim's browser into issuing a GET request to a vulnerable application
	POST CSRF	Forges POST request that executes a sensitive action on an authenticated victim
Cross-Site Scripting (XSS) [CWE-79]	DOM XSS	Injects malicious script on the client side that modifies the DOM with intrusted input
	Reflected XSS	Reflects malicious payload back to the victim in HTTP response
	Stored XSS	Stored payload in the server then executed in victim's browser
Untrusted Data Deserialization [CWE-502]	Insecure Deserialization	Blindly deserializes untrusted serialized data that triggers malicious behavior
Exposed internal object references [CWE-22]	Insecure Direct Object Reference (IDOR) Attack	Manipulation of exposed internal identifiers to gain unauthorized access to other users' data or actions.
Improper XML Parsing [CWE-611]	XXE Attack	Injects malicious external entities, enabling file disclosure, SSRF, or denial of service.
SQL Injection [CWE-89]	Blind SQLi	Infers information from the database relying on indirect responses
	Boolean-Based SQLi	Sends queries that return true/false conditions to deduce data
	Error-Based SQLi	Produces error messages that reveal details about the database structure and contents.
	Stacked Queries	Injects SQL statements in a single request, executing extra queries.
	Time-Based SQLi	Use delays (e.g., SLEEP) to infer data based on the time the server takes to respond
Unrestricted URL fetching [CWE-918]	Union-Based SQLi	Uses UNION to append malicious queries for data retrieve from other tables.
	Server-Side Request Forgery (SSRF)	The server blindly requests user-supplied URLs, allowing attackers to access internal systems or sensitive data.
Unsafe object property merging [CWE-1321]	Prototype Pollution	Injects or overrides properties in an object's prototype leading to unexpected behavior, data manipulation or even remote code execution.
Unsanitized file path input [CWE-98]	Local File Inclusion (LFI)	Trick the server into including local system files, exposing sensitive data or enabling code execution.
Improper Input Sanitization [CWE-77]	Command Injection	Executes arbitrary commands on the host system.

TABLE III
SCRIPTS GENERATION STATISTICS

	Benign	Malicious	Total
Generated	7543	5917	13460
Cleaned	2063	1617	3680
Final JS scripts	5480	4300	9780

- **Vulneraility:** the weakness that can be exploited by the script sample to gain unauthorized access or cause harm.
- **Attack:** the harmful action caused by the script.

The summary of our dataset statistics are detailed on the Table IV to facilitate reproducibility. This includes generated script counts grouped by obfuscation and shot strategy types of the final sample sizes after data cleaning.

The dataset is available through the following link: [16]

B. Classification

To evaluate the quality of our snippets and assess their utility in AI-driven applications, we developed a classifier by finetuning the MicroSoft pretrained codeBERT LLM. A foundation model -with about 125 million parameters and trained on a multi-lingual corpus of six programming languages including JavaScript- that has proven effectiveness in numerous recent works on software vulnerability and defect detection [17]. Moreover, the model includes a task-specific classification head, constructed with Classify token representation, Feed-Forward connected linear layer, and Softmax and Sigmoid activation functions.

We trained the model on our synthetic dataset by selecting hyperparameters based on common finetuning practices for medium databases. The dataset was divided into three distinct subsets, balanced according to the procedure's objective. A 70% allocated to provide the model with sufficient data for learning, 15% to ensure the validation of the training process, and the remaining 15% for testing to assess its reliability.

- learning rate: $2e - 5$
- batch size: 12
- epochs: 20

To evaluate the intrinsic quality and diversity of the synthetic scripts, we applied a cross-dataset evaluation strategy with the same hyperparameters (and batch size 48). The model was initially finetuned on *Sschumat* [18] dataset -composed of 48782 scripts (9333 benign and 39449 malicious)- then tested on our synthetic scripts, where the comparison provides insight into effectiveness of the generated snippets. For the

TABLE IV
SYNTHETIC DATASET STATISTICS

Label	Obfus.	0 shot	1 shot	2 shot	S.total	Total
Benign	✗	1128	762	831	2721	5480
	✓	558	899	1302	2759	
Malicious	✗	1117	467	440	2024	4300
	✓	563	1155	558	2276	

second experiment, and to validate the impact of artificial scripts we reversed the process by finetuning the model on our dataset, where we were able to predict 10k samples of the *Sschumat* snippets. The performance of both scenarios is assessed using accuracy, precision, recall, and F1-score to evaluate the effectiveness of classification and the balance between false positives and false negatives.

Table V shows the classification quantitative comparing performance asymmetries between the two datasets. The evaluation of predictions using the finetuned model in our scripts reveals a nuanced insight of its strengths and limitations. On one side, the high prediction accuracy of *splet* samples confirms that the generation and labeling processes are effective for more than 99% of cases and the classifier reliably distinguishes between harmful and benign scripts. Moreover, Table VI demonstrates the predictions of *Sschumat* scripts where the model shows promising performance with a balanced accuracy of 0.69, where 21% of the samples were misclassified (Fig 4). On the other side, the classifier finetuned on *Sschumat*'s dataset, shows that prediction of benign samples was suboptimal. However, it failed to recognize the majority of synthetic harmful scripts. Despite demonstrating high finetuning performance.

Sschumat's snippets do not show complex structures such as obfuscation heterogeneity or attacks diversity, which affects the recognition ability of the majority of our synthetic samples, more precisely obfuscated ones. As a result, the finetuned model was unable to correctly classify complex variants.

C. Limitations and Future Work

While the synthetic dataset demonstrated promising results, there still notable improvements to bypass meta-text behavior and refusal direction problems, while inferring for a task

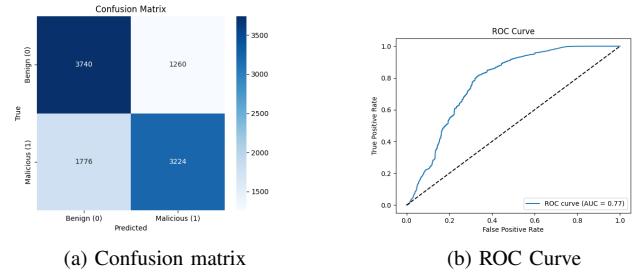


Fig. 4. Synthetic dataset classification graphs

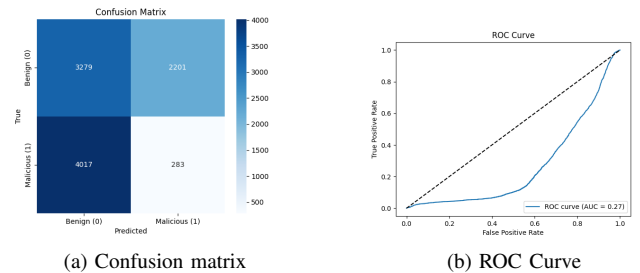


Fig. 5. *Sschumat* dataset classification graphs

TABLE V
CLASSIFICATION ON TEST SPLIT DATASET

	Synthetic dataset				Sschumat dataset			
	Precision	Recall	F1 score	Support	Precision	Recall	F1 score	Support
Benign	0.9963	0.9909	0.9936	551	0.9953	0.9965	0.9959	868
Malicious	0.9883	0.9953	0.9918	427	0.9992	0.9990	0.9991	4011
Accuracy	0.9928	0.9928	0.9928	0.9928	0.9985	0.9985	0.9985	0.9985
Macro avg	0.9923	0.9931	0.9927	978	0.9973	0.9977	0.9975	4879
Weighted avg	0.9928	0.9928	0.9928	978	0.9985	0.9985	0.9985	4879

TABLE VI
CLASSIFICATION ON SYNTHETIC AND SSCHUMAT DATASETS

	Sschumat dataset classified on Synthetic training				Synthetic dataset classified on Sschumat training			
	Precision	Recall	F1 score	Support	Precision	Recall	F1 score	Support
Benign	0.6780	0.748	0.7112	5000	0.4494	0.5983	0.5133	5480
Malicious	0.7190	0.6448	0.6798	5000	0.1139	0.0658	0.0834	4300
Accuracy	0.6964	0.6964	0.6964	0.6964	0.3642	0.3642	0.3642	0.3642
Macro avg	0.6985	0.6964	0.6955	10000	0.2816	0.3320	0.2983	9780
Weighted avg	0.6985	0.6964	0.6955	10000	0.3019	0.3942	0.3243	9780

specific code generation. Moreover, validating resulted scripts needs correctness analysis. Thereby, to overcome those challenges, we plan to develop a sophisticated generative model leveraged with dynamic malware analysis tools. Furthermore, for a fair comparison and data evaluation, we need to address scarcity of real-world heterogeneous labeled snippets. Thus, we might pre-process an existing dataset by labeling samples and extracting significant indicators for an advanced finetuning approach.

V. CONCLUSION

In this paper, we collected a synthetic dataset using large language models generation for AI-driven malware analysis applications, constructed of labeled benign and malicious JavaScript heterogeneous samples, and providing further details for each script. The effectiveness of the synthetic scripts was proved via prediction mechanism by finetuning codeBERT's model classifier head. The dataset demonstrates sub-optimal yet promising predictive performance on real-world scripts, highlighting both current limitations and potential directions for future research.

Ethical Statement

The synthetic dataset is provided strictly for academic research and using it for any malicious, fraudulent, or illegal activity is expressly prohibited. The creators and distributors of this dataset assume no responsibility for any misuse of the scripts or for any damages resulting from its use.

REFERENCES

- [1] M. F. Rozi, T. Ban, S. Ozawa, A. Yamada, T. Takahashi, S. Kim, and D. Inoue, "Detecting malicious javascript using structure-based analysis of graph representation," *IEEE Access*, vol. 11, pp. 102 727–102 745, 2023.
- [2] P. Ladisa, S. E. Ponta, N. Ronzoni, M. Martinez, and O. Barais, "On the feasibility of cross-language detection of malicious packages in npm and pypi," in *Proceedings of the 39th Annual Computer Security Applications Conference*, 2023, pp. 71–82.
- [3] N. M. Phung and M. Mimura, "Detection of malicious javascript on an imbalanced dataset," *Internet of Things*, vol. 13, p. 100357, 2021.
- [4] Y. Fang, C. Huang, M. Zeng, Z. Zhao, and C. Huang, "Jstrong: Malicious javascript detection based on code semantic representation and graph neural network," *Computers & Security*, vol. 118, p. 102715, 2022.
- [5] S. K. Shandilya, C. Ganguli, I. Izonin, and A. K. Nagar, "Cyber attack evaluation dataset for deep packet inspection and analysis," *Data in Brief*, vol. 46, p. 108771, 2023.
- [6] M. Silva, D. Pinto, J. Vitorino, J. Gonçalves, E. Maia, and I. Praça, "Genis: A modular dataset for network intrusion detection and classification," *Data in Brief*, vol. 60, p. 111487, 2025.
- [7] M. Ishida, N. Kaneko, and K. Sumi, "Moji: Character-level convolutional neural networks for malicious obfuscated javascript inspection," *Applied Soft Computing*, vol. 137, p. 110138, 2023.
- [8] D. K. Kholgh and P. Kostakos, "Pac-gpt: A novel approach to generating synthetic network traffic with gpt-3," *IEEE Access*, vol. 11, pp. 114 936–114 951, 2023.
- [9] A. Razmyslovich, K. Murasheva, S. Sedlova, J. Capitaine, and E. Dmitriev, "Eltex: A framework for domain-driven synthetic data generation," *arXiv preprint arXiv:2503.15055*, 2025.
- [10] M. M. Yamin, E. Hashmi, M. Ullah, and B. Katt, "Applications of llms for generating cyber security exercise scenarios," *IEEE Access*, 2024.
- [11] D. A. Ammara, J. Ding, and K. Tutschku, "Synthetic network traffic data generation: A comparative study," *arXiv preprint arXiv:2410.16326*, 2025.
- [12] Y. M. Pa Pa, S. Tanizaki, T. Kou, M. Van Eeten, K. Yoshioka, and T. Matsumoto, "An attacker's dream? exploring the capabilities of chatgpt for developing malware," in *Proceedings of the 16th cyber security experimentation and test workshop*, 2023, pp. 10–18.
- [13] A. Monje, A. Monje, R. A. Hallman, and G. Cybenko, "Being a bad influence on the kids: Malware generation in less than five minutes using chatgpt," *J. Cybersecur. Res*, 2023.
- [14] M. Du, A. T. Luu, B. Ji, X. Wu, D. Huang, T. Y. Zhuo, Q. Liu, and S.-K. Ng, "Codearena: A collective evaluation platform for llm code generation," *arXiv preprint arXiv:2503.01295*, 2025.
- [15] J. Li, G. Li, X. Zhang, Y. Zhao, Y. Dong, Z. Jin, B. Li, F. Huang, and Y. Li, "Evocodebench: An evolving code generation benchmark with domain-specific evaluations," *Advances in Neural Information Processing Systems*, vol. 37, pp. 57 619–57 641, 2024.
- [16] H. Ikni and A. E. Belfedhal, "Js synthetic dataset," <https://github.com/Hind-Ikni/JS-Synthetic-Dataset>, 2025.
- [17] R. G. Hussain, K.-C. Yow, and M. Gori, "Leveraging an enhanced codebert-based model for multiclass software defect prediction via defect classification," *IEEE access*, 2025.
- [18] Sschumat, "Malicious and benign javascript dataset," <https://www.kaggle.com/datasets/sschumat/malicious-and-benign-javascript-dataset>, accessed: Jul. 20, 2025.